Lightweight In-Network Flow Classification with Deep Differentiable Logic Gate Networks

Song Liu, Kaiwei Gao, Xinjing Yuan, Jianxin Shi, Lingjun Pu
College of Computer Science, DISSec, Nankai University, Tianjin, China
The corresponding author is Xinjing Yuan (yuanxinjing@nankai.edu.cn)

2025 IEEE 31th International Conference on Parallel and Distributed Systems (ICPADS) | 979-8-3315-4901-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICPADS67057.2025.11323087

***Abstract*—Deploying artificial intelligence (AI) models on the programmable data plane is a key direction toward realizing intelligent data planes. However, this vision faces significant challenges: existing approaches often incur excessive consumption of scarce switch hardware resources or introduce packet recirculation, leading to performance bottlenecks. To address these issues, we propose SwitchLGN, a novel Deep Differentiable Logic Gate Network (DDLGN) architecture deployable on programmable switches. The core of SwitchLGN lies in its hardware-aligned design, which decomposes the model into multiple independent sub-layers and maps each to distinct pipeline processing units. This design enables the entire inference process to be executed solely with the switch's native bit-level logic operations, thereby eliminating the challenges of cross-cluster computation and complex arithmetic in the data plane. In addition, we develop a compilation toolchain to support the automated mapping of SwitchLGN models to P4 code. Experimental evaluations show that SwitchLGN achieves accuracies of 99.42% for network anomaly detection and 95.59% for flow size classification, while reducing SRAM usage to below 3% and making TCAM consumption negligible. Notably, it delivers line-rate inference without packet recirculation, achieving a per-packet latency of only 283 ns.**

***Index Terms*—Programmable Network, In-network Computing, Deep Differentiable Logic Gate Network, Flow Classification.**

I. Introduction

The emergence of the Programmable Data Plane (PDP) enables network devices to achieve flexible programming of packet processing logic while maintaining line-rate forwarding performance [1], [2]. Deploying artificial intelligence (AI) technologies such as machine learning and neural networks on the PDP further promotes the intelligence of network devices. Such schemes combine the PDP's line-rate packet processing with AI-based traffic classification, enabling network decision making and improving service quality.

Studies [3]–[11] have attempted to offload AI models, such as decision trees (DT) [3]–[7], MLP [12], BNN [8], and CNN [9] to the programmable data plane. These studies have preliminarily verified the feasibility and potential applications of deploying lightweight AI models on data planes. However, most existing research fails to consider the resource constraints of the PDP, as it is required not only to perform intelligent functions but also to maintain its original packet forwarding capacity in practical network environments [7], [13]. Therefore, it should reserve sufficient resources for its core traditional functions and dedicate part to AI inference, while sustaining line-rate performance.

Existing in-network AI models, such as CNNs [9] and DTs [8], [14], often incur high resource costs, requiring extensive SRAM, costly TCAM, and packet recirculation. This hinders the performance of critical network functions like the 5G UPF [10], [15] and data center gateways, while attempts to reduce resource usage typically sacrifice model accuracy [4]. To address these challenges, we adopt the Deep Differentiable Logic Gate Network (DDLGN) [16], [17]. DDLGN performs inference solely through bitwise operations (e.g., AND, OR), a design that aligns naturally with the native capabilities of programmable switches. This approach eliminates complex arithmetic, thereby minimizing computational and memory overhead (SRAM/TCAM) while, as shown in prior work [17], maintaining accuracy comparable to more complex models.

The original DDLGN topology is incompatible with programmable switches due to PHV size, cluster-local computation, and compiler constraints. We design SwitchLGN, a hardware-aligned variant that partitions each layer into sub-layers on separate PHV clusters, ensuring all operations are local and avoiding cross-cluster dependencies. In this design, each neuron occupies a dedicated PHV container, features are bit-sliced at parsing, and per-stage ALUs execute boolean operations. Local votes are aggregated via popcount and combined globally (using a resubmit on Tofino1 or a single pass on Tofino2). This enables a recirculation-free deployment with predictable, low resource usage, primarily relying on PHV with minimal SRAM and negligible TCAM.

The contributions of this paper are as follows:

1) To the best of our knowledge, this is the first work to introduce DDLGN into the PDP and implement their deployment on commercial programmable switches. We demonstrate the feasibility of applying DDLGN for in-network computation.
2) Based on the hardware characteristics of programmable switches, we design SwitchLGN, a compact and deployment friendly DDLGN architecture that achieves line-rate inference on programmable switches without recirculation and with low resource overhead, all without sacrificing model accuracy.
3) We develop a complete compilation toolchain that supports efficient and automatic mapping from SwitchLGN models to P4 programs.

Experimental results on two representative flow classification tasks, network intrusion detection using the UNSW-NB15

dataset and flow size classification using the UNIV1 dataset, demonstrate that SwitchLGN attains accuracies of 99.42% and 95.59%, respectively. The accuracy is reduced by no more than 0.4% compared with the original DDLGN, while still surpassing state-of-the-art in-network models such as Quark (CNN) and Planter (DT). In addition, SwitchLGN lowers SRAM consumption to 2.6%, completely avoids TCAM usage, and sustains line-rate processing at 99.8 Gbps with an average per-packet inference latency of only 283 ns.

II. Background and Motivation

A. Programmable Switch

Tofino programmable switch [18] is a representative hardware of PDP. It typically employs a pipeline design based on the Protocol Independent Switch Architecture (PISA) [19], with packet processing workflows defined in the P4 language [20], enabling protocol independence. In a typical process, when a packet enters the switch, the parser first extracts key fields and populates them into the Packet Header Vector (PHV). The packet is then passed through a stage-based pipeline, where each stage consists of multiple Match-Action Units (MAUs) that perform field matching and corresponding actions. Finally, the deparser reassembles the packet and completes forwarding.

Although the PISA architecture enables line-rate processing, its programmability is constrained by hardware resources [21]. A pipeline consists of multiple stages with SRAM/TCAM for lookups and ALUs for computation, and each pipeline in Tofino 1 and 2 contains 12 and 20 stages, respectively [18]. Total on-chip memory is limited to tens of megabytes, with SRAM used for exact matches and the smaller, costlier TCAM for ternary matches. Registers and PHV containers are stage-bound and accessible only once per stage, so complex operations must be split across stages or implemented via packet recirculation or table lookups. Each expression can use at most two variables in the same PHV cluster, further limiting the flexibility.

Programmable switches support only a limited set of integer arithmetic and logic operations, such as addition, subtraction, and bit shifts, while lacking native support for floating-point operations and division. Applications requiring floating-point arithmetic must use fixed-point approximations and table lookup for complex multiply accumulate operations. These constraints make the implementation of complex AI algorithms on switches require many pipeline stages, registers, and tables, consuming memory resources and potentially reducing throughput and accuracy.

B. AI in Programmable Data Plane

Research on AI in programmable switches follows two main lines: tree-based models and NNs. Tree-based models like DTs and Random Forests found early adoption as their structure aligns with the match–action pipeline. However, their depth—and thus complexity—is limited by available MAUs. Later works [5] used encoding (e.g., mapping tree levels to tables) to reduce pipeline depth. While this lowers the per-stage burden, it creates high memory demand, as deeper trees require more entries and thus consume more SRAM and TCAM. Lightweight NNs can be deployed on PDPs, such as MLP [12], BNN [8], CNN [9], and RNN [22]. However, PISA-based switches only support bit-wise operations, table lookup, and simple integer arithmetic. Operations such as multiplication and normalization in NNs must be approximated through quantization [9], [12], binarization [8], or precomputed tables. Quantization and binarization reduce computation but can harm accuracy and model capacity [14]. Table-based mapping preserves accuracy but increases memory consumption. For example, the P4 implementation of N3IC [8] adds about 22% SRAM for batch normalization and lookup, and Quark [9] stores all convolution products in SRAM to avoid multiplications, resulting in a 22.7% overhead.

Existing in-network models, including decision trees, BNNs, and quantized neural networks, emulate unsupported arithmetic operations on programmable switches through large TCAM or SRAM tables, resulting in substantial memory overhead. As shown in Table I, none of these models can avoid heavy reliance on scarce memory while maintaining both high accuracy and line-rate inference, which motivates the design of our work.

C. Deep Differentiable Logic Gate Network

To address the above issues, we introduce a logic gate based inference model, the Deep Differentiable Logic Gate Network as a new approach for in-switch data-plane intelligence. As shown in Fig. 1, DDLGN builds each neuron from basic boolean gates (e.g., AND, OR, NOT). Each neuron receives binary inputs, and through interconnections among neurons, the network as a whole achieves nonlinear mappings. This design also combines real valued logic with a continuous, parameterized relaxation to enable effective training. The inference requires no floating-point arithmetic and no multiplications, but instead consists solely of bit-wise operations. Such operations are natively supported by programmable switches and can be directly mapped to the data path. DDLGN has also been shown effective for image classification [16], [17].

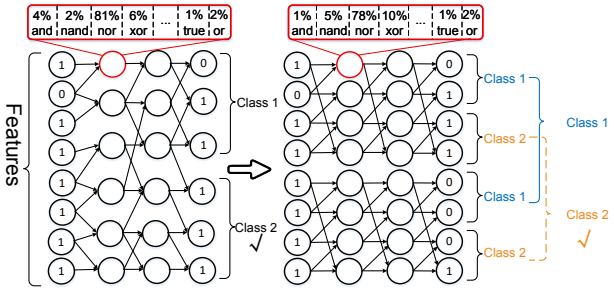


Fig. 1: Original DDLGN (left) with random inter-layer connections and the proposed SwitchLGN (right) with fixed intra-sub-layer connections aligned to PHV clusters.

TABLE I: In-Network ML Approaches: A Comparison of Computation and Hardware Costs.

In-Network ML	Core Computation	Major Hardware Overhead	Representative Works
Decision Tree	Encoded table lookup	Moderate SRAM or TCAM	IIsy [3], Planter [5]
Binary Neural Network (BNN)	Xnor-popcount-based bitwise multiplication	Moderate SRAM, Negligible TCAM	N3IC [8]
Quantized Neural Network	Lookup-table-based multiplication	Extremely High SRAM, Negligible TCAM	Quark [9], INQ-MLT [12]
DDLGN	Native bit-level logic operations	Negligible SRAM & TCAM	SwitchLGN (our)

Despite its natural affinity to bit-wise architectures, the original DDLGN is not directly deployable on programmable switches. DDLGN layers produce and pass many intermediate variables, but the PHV is limited in size and cannot hold all intermediate results for larger models. The pipeline has a fixed number of stages, and each stage supports only a limited set of logical operations and a limited number of field writes. Additional, compiler constraints, such as PHV field dependency order, operator limits, and action complexity, further prevent a direct mapping of the original DDLGN to P4 code. Therefore, the key challenge is to redesign an DDLGN that fits programmable switches without sacrificing too much inference capability, and to map it into P4 code under these hardware constraints.

III. System Overview

As shown in Fig. 2, the SwitchLGN framework comprises two main components: the control plane and the data plane. This framework is built upon SwitchLGN, a hardware-aligned variant of the DDLGN designed for efficient deployment on programmable switches. SwitchLGN retains the inference principles of DDLGN while restructuring its topology to match hardware constraints such as PHV size, cluster-local computation, and limited pipeline stages.

The control plane is responsible for collecting traffic data from the data plane or developer uploads, preprocessing the data, and training the SwitchLGN model. Once trained, the model is compiled into P4 code with hardware-aware optimizations, including sub-layer partitioning and PHV cluster alignment. The compiled code is deployed to the programmable switch via control-plane APIs such as P4Runtime, BFRuntime, or gRPC using fast reconfiguration [23]. Although deployment temporarily pauses packet processing, the update latency is typically below 50 ms [23], which is negligible for most online services.

The data plane executes the trained SwitchLGN model directly on the programmable switch. Incoming traffic features, including five-tuples, port numbers, packet lengths, and protocol identifiers, are mapped into PHV containers. Multi-bit fields are decomposed into individual 1-bit features through assignment and bit-shift operations to satisfy the binary input requirement of SwitchLGN. Each pipeline stage implements one model layer, performing the associated boolean computations and writing the results back into PHV containers. After all layers are processed, a bit-wise popcount aggregates neuron activations into per-class vote counts, and the final label is determined by comparing these counts. Based on the classification result, the switch applies the appropriate forwarding policy, such as redirecting high-risk flows to the CPU for further analysis or forwarding benign flows directly.

IV. SwitchLGN Model Design

A. Model Architecture

Considering that programmable switches organize PHVs into clusters and perform computations locally within the same cluster, we redesign the original DDLGN architecture into a hardware-aligned architecture. Each layer is partitioned into multiple independent sub-layers, and a fixed, localized wiring pattern is employed to match the "intra-cluster local computation" data path. This subsection defines only the model's connection scheme, while the detailed implementation mapping is deferred to the implementation section. The neuron connectivity is defined as follows.

Let the number of neurons in layer l be N_l. The layer is divided into S sub-layers, each containing $n_l = N_l/S$ neurons. We assume that all layers have the same total number of neurons and that all sub-layers contain the same number of neurons. Denote by $h_l \in \{0,1\}^{N_l}$ the output vector of layer l, and by $h_l^{(s)} \in \{0,1\}^{n_l}$ the output vector of the s-th sub-layer, where $s \in \{0,\dots,S_l-1\}$ and $j \in \{0,\dots,n_l-1\}$ is the position index within the sub-layer.

We adopt a fixed topology where, within each sub-layer, every neuron takes two inputs from adjacent neurons. The j-th neuron of the s-th sub-layer in layer $l+1$ receives inputs only from the j-th and $(j+1) \bmod n_l$-th neurons of the same sub-layer in the preceding layer, and computes its output via a two-input boolean gate $\mathcal{G}_{l+1}^{(s)}(\cdot,\cdot)$:

$$h_{l+1}^{(s)}[j] = \mathcal{G}_{l+1}^{(s)}\left(h_l^{(s)}[j],\, h_l^{(s)}\left[(j+1) \bmod n_l\right]\right), \quad j = 0,\dots,n_l-1. \tag{1}$$

To improve the predictability of PHV allocation by the P4 compiler, the first layer of SwitchLGN uses a reduced boolean operator set $\mathcal{O}_{\text{thin}}$. This set contains 10 strictly binary logic gates, such as AND, OR, XOR, and their negated or composite variants, where the right-hand side of each operation explicitly involves two variables. The remaining layers use the full operator set $\mathcal{O}_{\text{full}}$ of 16 gates, including constants and unary negation. Those gates can be seen in Table II. The rationale is that, in practical compilation for programmable switches, when the first-layer operators always reference two variables and connections are restricted within sub-layers, the compiler is more likely to place both inputs and the write-back target in the same PHV cluster, thereby reducing the frequency and

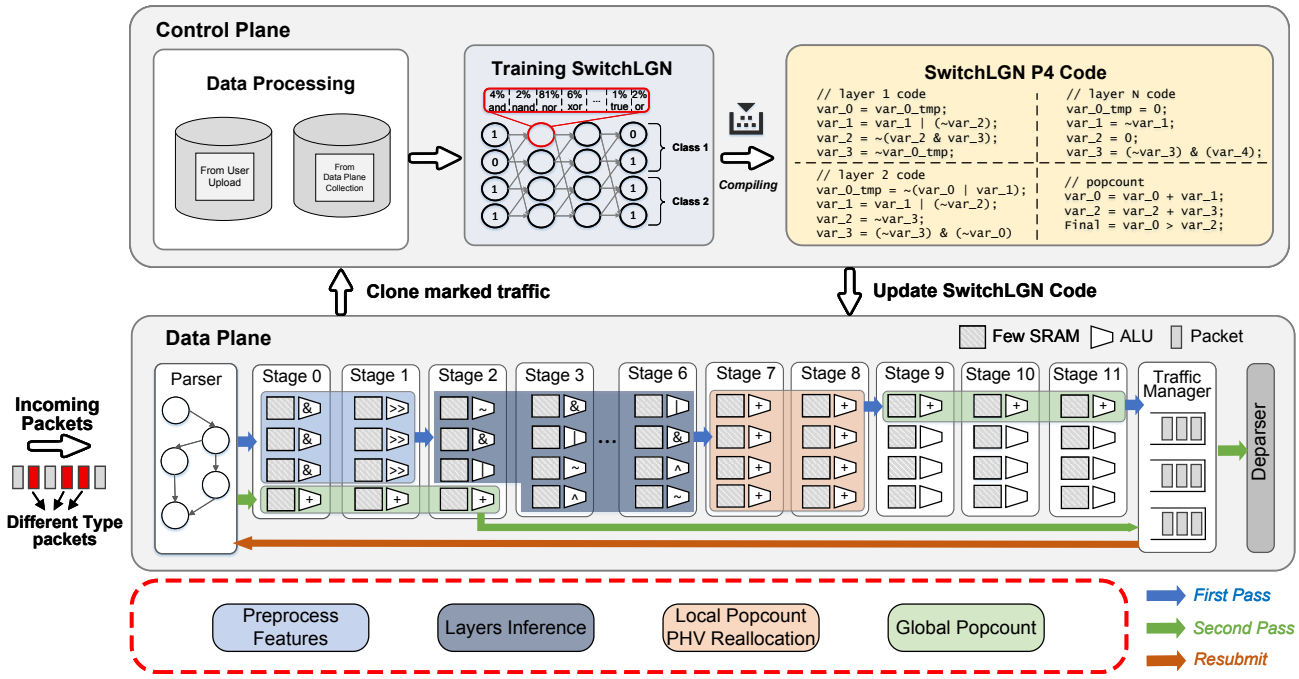


Fig. 2: Overall architecture of the proposed SwitchLGN Framework.

overhead of cross-cluster reallocation. In other words, this is an *implementation-aware* modeling choice aimed at improving the predictability of first layer PHV allocation. Experimental results show that under the evaluated tasks and model size, this constraint has negligible impact on classification accuracy.

The above neuron connectivity can be viewed as partitioning the overall model into multiple relatively independent sub-models that are trained and inferred in parallel. Each sub-layer is treated as one layer of an independent sub-model, producing local decisions in parallel during the feed-forward process. For a binary classification task, the final layer in each sub-layer produces two binary output vectors. Within each sub-layer, a bit-wise popcount is performed for each class to obtain local vote counts. These local counts are then aggregated across sub-layers along the same class dimension by summing the counts from all sub-layers to obtain the global vote counts, and the class with the highest global count is selected as the final prediction.

B. Feature Processing

The feature processing in SwitchLGN follows the bit-level computation paradigm: original fields are firstly bit sliced, and each bit is mapped to a single neuron in the input layer. To align with the PHV and cluster organization of the programmable switch and reduce the likelihood of cross-cluster binding during compilation, we adopt a *feature-to-sub-model* allocation strategy, whereby each feature is preferentially mapped to a single sub-model so that its boolean operations are performed within the same cluster, avoiding cross-cluster dependencies.

When a feature's bit width significantly exceeds the neuron capacity n_0 of a single input-layer sub-model, it is split at fixed boundaries into multiple consecutive bit segments, which are assigned to different sub-models for parallel processing. When the bit width slightly exceeds n_0, A constrained truncation policy is applied: n_0 bits are retained within a single sub-model. For example, if a sub-model contains 14 neurons and a feature is 16 bits wide, the lower 14 bits are taken as the input. As shown in our experiments, this localized mapping strategy maintains high feasibility and stable classification accuracy for the evaluated tasks and model size.

TABLE II: List of all real-valued binary logic gates.

ID	Operator	real-valued	$\mathcal{O}_{\text{full}}$	$\mathcal{O}_{\text{thin}}$
0	False	0	✓	
1	$A \wedge B$	$A \cdot B$	✓	✓
2	$\neg(A \Rightarrow B)$	$A - AB$	✓	✓
3	A	A	✓	
4	$\neg(A \Leftarrow B)$	$B - AB$	✓	✓
5	B	B	✓	
6	$A \oplus B$	$A + B - 2AB$	✓	✓
7	$A \vee B$	$A + B - AB$	✓	✓
8	$\neg(A \vee B)$	$1 - (A + B - AB)$	✓	✓
9	$\neg(A \oplus B)$	$1 - (A + B - 2AB)$	✓	✓
10	$\neg B$	$1 - B$	✓	
11	$A \Leftarrow B$	$1 - A + AB$	✓	✓
12	$\neg A$	$1 - A$	✓	
13	$A \Rightarrow B$	$1 - A + AB$	✓	✓
14	$\neg(A \wedge B)$	$1 - AB$	✓	✓
15	True	1	✓	

C. Model Training

Training is performed on the control plane, with the objective of selecting the optimal boolean operator for each neuron to minimize task loss. To enable gradient-based optimization for this discrete selection problem, we approximate boolean gates using their real-valued logic formulations. Each neuron

maintains a set of learnable parameters, which are passed through a softmax function to generate selection probabilities over the candidate gates. During forward propagation, these probabilities are used to compute a weighted sum of all gate outputs. The parameters are subsequently updated via gradient descent. After convergence, the gate with the highest selection probability at each neuron is chosen for the final model.

Specifically, let the two inputs to neuron $h_{l+1}^{(s)}[j]$ in the s-th sub-layer at position j of layer $l+1$ be $h_l^{(s)}[j]$ and $h_l^{(s)}[(j+1) \bmod n_l]$. The candidate boolean gate set for all neurons in layer $l+1$ is defined as

$$\mathcal{O}_{l+1} = \begin{cases} \mathcal{O}_{\text{thin}}, & l = 0 \\ \mathcal{O}_{\text{full}}, & l \geq 1, \end{cases} \tag{2}$$

where $\mathcal{O}_{\text{thin}}$ denotes the reduced set used in the first layer and $\mathcal{O}_{\text{full}}$ denotes the complete set used in all subsequent layers. For each candidate boolean gate $r \in \mathcal{O}_{l+1}$ of neuron $h_{l+1}^{(s)}[j]$, we assign a learnable floating-point parameter $\alpha_{l+1}^{(s,j,r)} \in \mathbb{R}$. The selection probability $\pi_{l+1}^{(s,j,r)}$ for gate r is obtained via the softmax function:

$$\pi_{l+1}^{(s,j,r)} = \frac{\exp\left(\alpha_{l+1}^{(s,j,r)}\right)}{\sum_{r' \in \mathcal{O}_{l+1}} \exp\left(\alpha_{l+1}^{(s,j,r')}\right)}. \tag{3}$$

To enable gradient-based optimization in SwitchLGN during forward propagation, each boolean gate function $G(\cdot,\cdot) : \{0,1\}^2 \to \{0,1\}$ is extended to a continuously differentiable function $g(\cdot,\cdot) : [0,1]^2 \to [0,1]$ over the real valued domain. The output of neuron $h_{l+1}^{(s)}[j]$ in the relaxed form is then computed as the weighted sum

$$\tilde{h}_{l+1}^{(s)}[j] = \sum_{r \in \mathcal{O}_{l+1}} \pi_{l+1}^{(s,j,r)} \cdot g_r(a,b) \in [0,1]. \tag{4}$$

The loss function is based on the task-specific cross-entropy and follows the same "intra-sub-layer counting + inter-sub-layer summation" aggregation logic to construct *soft votes* during training. For a K-class classification task, let the final layer be L and denote the class index by $k \in \{1,\ldots,K\}$. Let $\tilde{h}_L^{(s,k)}$ be the soft output vector of sub-layer s for class k. The local vote count $\tilde{c}_k^{(s)}$ for class k in sub-layer s, and the global vote count $\tilde{c}_k$ obtained by summing across all sub-layers, are given by

$$\tilde{c}_k^{(s)} = \sum_j \tilde{h}_L^{(s,k)}[j], \qquad \tilde{c}_k = \sum_{s=0}^{S_L-1} \tilde{c}_k^{(s)}. \tag{5}$$

The training loss adopts the standard softmax cross-entropy, defined as

$$\mathcal{L}_{\text{cls}} = \text{CE}\Big(y,\ \text{softmax}\big([\tilde{c}_1,\ldots,\tilde{c}_K]\big)\Big), \tag{6}$$

where y is the ground-truth label. After convergence, all floating-point parameters α and probabilities π are discarded. For each neuron $(l+1,s,j)$, the candidate boolean operator with the highest selection probability is chosen and fixed in the final model:

$$\begin{aligned} r^\star &= \arg\max_{r \in \mathcal{O}_{l+1}} \ \alpha_{l+1}^{(s,j,r)}, \\ h_{l+1}^{(s)}[j] &= G_{r^\star}\left(h_l^{(s)}[j],\ h_l^{(s)}\left[(j+1) \bmod n_l\right]\right). \end{aligned} \tag{7}$$

During inference, no real valued approximation is performed. Instead, the network is evaluated layer-by-layer using the fixed boolean gates. In each sub-layer, a popcount is applied to the binary output vector of each class to obtain local vote counts, which are then summed across sub-layers along the class dimension to produce global vote counts. The class with the highest global vote count is returned as the prediction. Since all nonlinearities are provided by the boolean gates, SwitchLGN requires no separate activation function in either training or inference. The only difference between the first layer and subsequent layers lies in the size of their candidate operator sets, while the training process remains identical.

V. Data Plane Implementation

A. PHV Allocation Strategy

To support computations between neurons in SwitchLGN, we adopt a one-to-one mapping strategy in which each PHV container stores the state of a single neuron, regardless of whether the nominal width of the container is 8, 16, or 32 bits. In this design, each container represents only a 1-bit neuron state. The benefits are threefold: (1) eliminating alignment and interference issues caused by multi-bit reuse within the same PHV container; (2) ensuring that each neuron is fully contained within a single PHV container, avoiding cross container slicing and recombination; and (3) facilitating PHV container reuse across different layers without requiring additional allocation.

Furthermore, we constrain all PHV containers used by a given sub-model to reside within the same PHV cluster. This is achieved by defining, in the switch `meta` fields, contiguous blocks for all sub-models (e.g., `sub_layer_x.var_y`) and keeping the field widths within each block consistent, which guides the compiler to pack them into the same PHV cluster. This strategy makes the compiler's resource allocation more predictable and the overall resource usage more controllable.

B. Feature Processing in Data Plane

To satisfy SwitchLGN's requirement for binary inputs and ensure PHV cluster locality during inference, each feature is converted into multiple 1-bit neurons, with each bit mapped independently to a dedicated PHV container within its corresponding sub-model. The entire mapping process is implemented in the data plane in three steps: feature assignment, bit selection, and right-shift alignment, which are executed in the parser and two consecutive pipeline stages, respectively.

First, during parsing, the required features are extracted and assigned. Multiple fields are obtained from the packet header, such as `diffserv`, `flags`, `total_len`, and `src/dst_addr`, representing key traffic characteristics. Each field is assigned to a predefined set of PHV containers

within a sub-model. For example, a 16-bit field is assigned to the 16 PHV containers corresponding to that sub-model. At this point, the values stored in these 16 containers are identical, each holding the original integer representation of the field.

The next two stages perform the extraction and alignment of bit-level neurons. In the first stage, a bit-wise AND operation is applied. Each PHV container is assigned a corresponding bit-mask that retains only the k-th bit of interest while clearing all other bits. Through this masking process, the first stage isolates each individual bit from the original field value and distributes them into 16 separate containers.

In the second stage, these masked containers undergo a bit-wise right shift. Each container is shifted by k positions, moving the target bit into the least significant bit position. After this operation, all container values are normalized to either 0 or 1, representing whether the original bit at that position was 0 or 1. This representation fully matches the input requirement of boolean logic gates, enabling subsequent boolean computations to be performed directly on these containers without any further alignment or masking.

C. Layers Inference

After feature processing, the PHV container values within each sub-model are bit-aligned and co-located in a single PHV cluster. Inference proceeds layer-by-layer, where each layer maps to a single pipeline stage and is decomposed into multiple independent sub-layers. Each sub-layer's logic is executed locally within its dedicated PHV cluster, guaranteeing data independence and avoiding cross-cluster interactions.

As described in §IV-A, each neuron in a sub-layer takes inputs from itself and the next position in a ring (*i.e.*, wrap-around at the end). However, this wiring creates a special case for `var_0`. Unlike most neurons, which allow safe in-place updates, such an update for `var_0` is hazardous. It would overwrite the original value before it is read by another operation within the same stage, creating a classic Read-after-Write hazard. This dependency violation is rejected by the compiler as it breaks PHV read/write rules.

To resolve this hazard, we employ a ping-pong update strategy. In any given stage, `var_0`'s computed result is written to a temporary container, `var_0_tmp`, while reads continue from the original `var_0`. In the subsequent stage, the roles of these two containers are swapped. This alternating, cross-stage update mechanism eliminates the conflict while maintaining full logical equivalence.

D. Popcount and Final Decision

After inference, we compute the final classification via a popcount. Because SwitchLGN uses a "one PHV container per 1-bit neuron" design, vote counting (popcount) can be implemented directly in the data plane. In our binary classification task setting where each layer of SwitchLGN contains 112 neurons evenly divided into 8 sub-layers, we assign the first 7 neurons of each sub-layer to class 1 and the remaining 7 to class 2. However, since each sub-layer is bound to a different PHV cluster, the local vote counts are physically separated, and the hardware does not support cross-cluster addition in one step. To efficiently address this challenge, we employ a table-driven method that integrates local counting with PHV redistribution. We use the 7 neuron activation values as a 7-bit key for a match-action table to look up their pre-computed sum. The action of this table not only returns the count but also writes it directly into a designated common PHV cluster.In this way, the entire process, from local counting and redistribution to the final global summation, is organized into a sequential computation that must span multiple pipeline stages.

Consider a binary, 5-layer SwitchLGN on Tofino 1. Due to the limited number of available pipeline stages, a single traversal cannot complete all local counts, redistribution, and the final global summation within 12 stages. We therefore trigger a `resubmit` at the end of the first traversal, carrying the redistributed counts into a second traversal, where the global adding and class decision are completed using the newly available stages. Unlike recirculation through a physical port, `resubmit` does not consume an external port for packet return and thus does not degrade throughput. On Tofino 2, where stage resources are more sufficient. Local counting, redistribution, and global aggregation can all be completed in a single pipeline pass, thereby reducing inference latency.

After obtaining the global vote counts, the data plane compares the two class totals and outputs the class with the larger count as the prediction. The classification result can be written into metadata for subsequent processing, e.g., used directly as a match key to steer flows and enforce priority scheduling in traffic classification. In security scenarios, it can trigger drop or quarantine actions. For traffic deemed malicious or anomalous, the data plane can also generate a report to the control plane, exporting contextual information (such as the five tuple and feature values) for more sophisticated analysis and response.

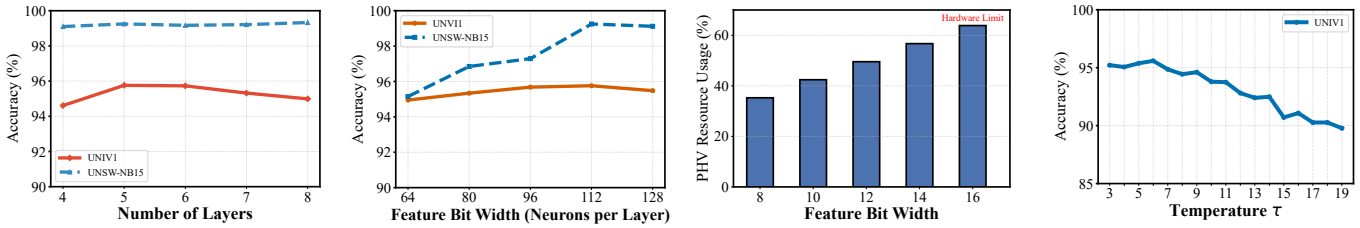
VI. Experiment

To validate SwitchLGN, we deployed it on a P4-programmable switch equipped with an Intel Tofino ASIC and conducted a series of experiments.

A. Experiment Setup

Our experimental testbed consists of two servers and a Flnet S9180-32X programmable switch equipped with an Intel Tofino 1 ASIC and 32×100 GbE ports. Each server is configured with an Intel Core i9-11900K processor, 64 GB RAM, and a 100 GbE network interface. We use Intel SDE v9.2.0 to compile $P4_{16}$ programs for deploying the SwitchLGN model onto programmable switch. In the experiments, one server acts as a traffic generator, transmitting traffic to the switch, while the other serves as the traffic sink. We employ the Pktgen [24] to measure and evaluate system throughput, and all models are trained offline on a separate GPU server equipped with an NVIDIA RTX 3080 Ti.

To comprehensively evaluate the performance of our SwitchLGN model under different network scenarios, we select two representative classification tasks, network intrusion detection and flow size classification:



(a) Impact of Network Depth (b) Impact of Network Width (c) PHV Usage per Width (d) Impact of Temperature τ

Fig. 3: Analysis of key hyperparameters for SwitchLGN, illustrating the impact of (a) network depth, (b) network width, and (d) training temperature τ on model performance. (c) shows the corresponding PHV consumption as network width increases.

- **Network Intrusion Detection**: We use the UNSW-NB15 dataset [25], generated by the IXIA PerfectStorm tool, which contains real normal network activity and nine types of modern network attack traffic. It is widely recognized as a benchmark for evaluating intrusion detection systems. We formulate this task as a binary classification problem to distinguish between Attack and Normal traffic. Eight key flow-level statistical features are extracted, and following standard evaluation practice, the dataset is split into 80% for training and 20% for testing.
- **Flow Size Classification**: To assess the model's capability in traffic prediction, we use the UNIV1 dataset [26] to classify flows into elephant flows and mice flows. Specifically, the top 20% of flows ranked by total bytes are labeled as elephant flows, and the remaining flows are labeled as mice flows. These labels serve as the ground truth for our binary classification task.

We compare SwitchLGN against a diverse set of baselines that represent different technical approaches in in-network machine learning.

1) **N3IC** [8]: A pioneering work that deploys BNN on programmable NIC.
2) **Quark&INQ-MLT** [9], [12]: State-of-the-art models that adapt traditional neural networks (CNN, MLP) to the data plane through quantization and other optimization techniques.
3) **IIsy&Planter** [3], [5]: They map traditional ML models such as decision trees and random forests onto programmable switches.
4) **LINC** [7]: A novel paradigm that interprets a neural network as a set of efficient match–action rules.

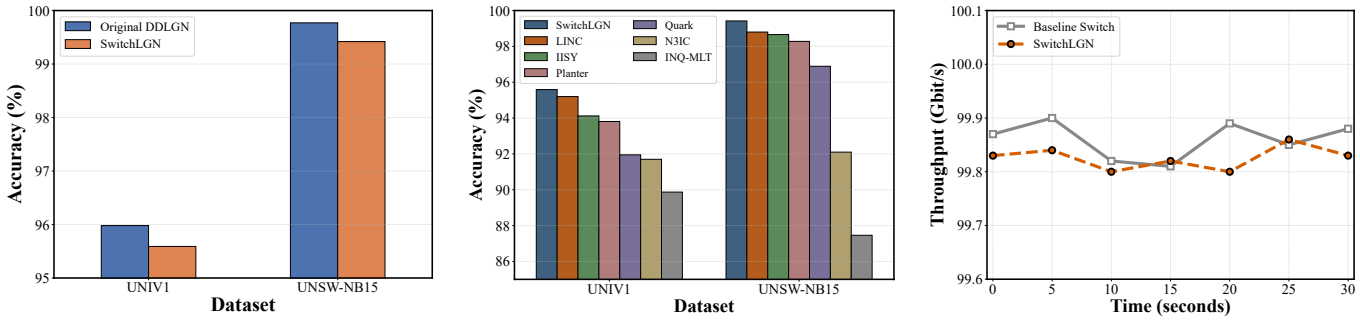
B. Parameters Affecting SwitchLGN

We conduct a series of studies to investigate the impact of model hyperparameters on the final performance and to guide the selection of the optimal model configuration.

1) Impact of Network Depth: In this experiment, we fix the network width (number of neurons per layer) at 112 and the temperature parameter τ at 6, while gradually increasing the network depth from 4 to 8 layers to observe its performance variation on the two tasks. As shown in Fig. 3a, in the UNIV1 flow size classification task, the model's performance exhibits clear sensitivity to network depth. When the number of layers increases from 4 to 5, the accuracy improves significantly from 94.61% to 95.59%, indicating that increasing depth within a certain range can enhance the model's ability to capture complex flow features. However, when the depth is further increased to 6 layers and beyond, all performance metrics begin to show a slight but consistent decline. In contrast, on the UNSW-NB15 intrusion detection task, the model follows a different trend, achieving its best performance with an 8-layer network. Nevertheless, in hardware deployment, each additional network layer occupies a valuable hardware pipeline stage and directly leads to a linear increase in inference latency. Therefore, a trade-off between accuracy and resource efficiency must be made. Considering all factors, the 5-layer configuration offers the best overall balance between performance and cost.

2) Impact of Network Width: In this experiment, we fix the network depth at 5 layers and the temperature parameter τ at 6 to investigate the impact of feature width (number of neurons per layer). The width is set to 8, 10, 12, 14, and 16 neurons per sub-layer. As shown in Fig. 3b, increasing the network width consistently improves model accuracy. In the UNIV1 task, when the number of neurons increases from 64 to 112, the accuracy rises steadily from 94.95% to a peak of 95.59%. A similar trend is observed on the UNSW-NB15 dataset, with the best performance also achieved at 112 neurons. This indicates that a wider network can capture feature information with greater granularity. However, this performance gain comes at a significant hardware cost. As shown in Fig. 3c, PHV consumption increases linearly with network width, since in our design, the computation result of each neuron must be temporarily stored in the PHV. When the width reaches 16, PHV usage hits the hardware limit. In contrast, SRAM and TCAM consumption remains minimal and unchanged. Therefore, 112 neurons per layer offer the best trade-off between accuracy and hardware feasibility.

3) Impact of Training Temperature τ: Based on the optimized architecture determined from the previous experiments, which consists of 5 layers with 112 neurons per layer, we further investigate the impact of a key training hyperparameter: the temperature coefficient τ. During training, τ controls the randomness in logic gate selection, directly affecting whether



(a) Accuracy comparison between the original DDLGN and our SwitchLGN.

(b) Performance comparison with other data plane models on UNIV1 and UNSW-NB15.

(c) Throughput comparison with a baseline switch, showing near line-rate performance.

Fig. 4: Data plane performance of SwitchLGN.

the model can converge to a high-performance discretized logic structure. As shown in Fig. 3d, we evaluate multiple τ values ranging from 3 to 19 on the UNIV1 dataset. The results indicate that the model's performance is highly sensitive to τ, with a clear optimal range. When $\tau = 6$, the model achieves its peak performance, reaching an accuracy of 95.59%. Therefore, based on these comprehensive experimental results, we select $\tau = 6$ as the final training parameter for our model.

C. Data Plane Performance

Based on these experiments, the final model adopts 5 layers, 112 neurons per layer, and a training temperature τ of 6 for performance benchmarking.

1) Accuracy: Classification accuracy is the primary metric for evaluating model performance. Before benchmarking against other state-of-the-art approaches, we first need to verify that our hardware-adapted SwitchLGN topology does not significantly compromise the theoretical accuracy of the original DDLGN model.

To this end, we conduct a key internal comparison experiment, evaluating both models under the same optimal hyperparameters. The results are shown in Fig. 4a. Compared to the original DDLGN, SwitchLGN exhibits only a negligible accuracy difference of approximately 0.4% across both datasets. This minimal performance change strongly demonstrates that our design represents an effective trade-off: it resolves the core challenge of efficiently deploying the original topology on the data plane without significantly sacrificing model accuracy. Having established the high fidelity of SwitchLGN, we further benchmark it against state-of-the-art AI model deployment schemes for the data plane. As shown in Fig. 4b, our SwitchLGN model demonstrates outstanding performance, achieving a classification accuracy of 99.42% on the UNSW-NB15 intrusion detection task and 95.59% on the UNIV1 flow size classification task. This performance significantly surpasses that of traditional models adapted for data plane deployment, such as decision trees and quantized neural networks, clearly demonstrating the advantage of our logic-gate-based approach in maintaining high accuracy for binary classification.

2) Throughput and Latency: Throughput is also a core metric for in-network computing. As shown in Fig. 4c, the programmable switch deployed with the SwitchLGN model achieves a line-rate processing throughput of 99.8 Gbps, almost identical to the 100 Gbps performance of a Baseline Switch, which represents a simple forwarding application without any deployed model. Inference latency refers to the delay from when a packet enters the switch and begins inference to when inference is completed and the packet is forwarded. For the SwitchLGN model, the average per-packet inference latency is only 283 ns. This nanosecond-level delay confirms that its inference process has a negligible impact on packet forwarding.

D. Resource Usage

The resource usage of SwitchLGN is shown in Table III. A distinctive aspect of its consumption is that it primarily relies on the PHV to temporarily store bit-level computation results between layers, occupying approximately 56.7% of the PHV space. Crucially, the inference process of SwitchLGN makes almost no use of costly TCAM resources, and its SRAM usage is also minimal (less than 3%). This stands in sharp contrast to traditional in-network computing approaches that depend heavily on large-scale table lookups (TCAM/SRAM), as illustrated in Table IV. This not only demonstrates the efficiency of SwitchLGN itself but also means it can easily coexist with other TCAM/SRAM-intensive applications on the switch without causing resource contention.

TABLE III: Resource utilization breakdown

Computational		Memory		PHV	
eMatch xBar	7.68%	SRAM	2.60%	8 b used	70.31%
tMatch xBar	0.38%	TCAM	0.00%	16 b used	59.38%
Gateway	0.00%	Map RAM	0.00%	32 b used	23.44%
VLIW	6.25%	TableID	13.02%		
Hash bits	4.40%	Stash	11.46%	Overall	56.70%

VII. Conclusion

We propose SwitchLGN, a logic-gate network architecture tailored for programmable switches, addressing the dual chal-

TABLE IV: Memory usage comparison on the UNIV1 dataset.

Model	SRAM Usage	TCAM Usage
Planter (DT)	6.04%	0.00%
IISY (DT)	3.20%	80.37%
Quark	24.27%	0.00%
SwitchLGN	2.60%	0.00%

lenges of high resource consumption and limited performance in existing in-network computing models. By decomposing the network structure in accordance with switch hardware characteristics, the inference process is fully mapped to native bit-level logic operations, enabling high-accuracy, nanosecond-latency, line-rate classification without recirculation. Experimental results demonstrate that SwitchLGN significantly reduces SRAM and TCAM usage while maintaining excellent inference performance, offering a practical solution for building efficient and cost-effective intelligent data planes. Future work will explore the deployment of SwitchLGN on other programmable hardware platforms and enhance its classification accuracy and applicability to broader tasks.

ACKNOWLEDGEMENT

This work is supported by the National Natural Science Foundation of China (No. 62172241, No. 62402247, No. 62502239).

REFERENCES

[1] O. Michel, R. Bifulco, G. Rétvári, and S. Schmid, "The programmable data plane: Abstractions, architectures, algorithms, and applications," *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–36, 2021.

[2] F. Hauser, M. Häberle, D. Merling, S. Lindner, V. Gurevich, F. Zeiger, R. Frank, and M. Menth, "A survey on data plane programming with p4: Fundamentals, advances, and applied research," *Journal of Network and Computer Applications*, vol. 212, p. 103561, 2023.

[3] C. Zheng, Z. Xiong, T. T. Bui, S. Kaupmees, R. Bensoussane, A. Bernabeu, S. Vargaftik, Y. Ben-Itzhak, and N. Zilberman, "IIsy: Hybrid in-network classification using programmable switches," *IEEE/ACM Transactions on Networking*, vol. 32, no. 3, pp. 2555–2570, 2024.

[4] G. Xie, Q. Li, Y. Dong, G. Duan, Y. Jiang, and J. Duan, "Mousika: Enable general in-network intelligence in programmable switches by knowledge distillation," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1938–1947.

[5] C. Zheng, M. Zang, X. Hong, L. Perreault, R. Bensoussane, S. Vargaftik, Y. Ben-Itzhak, and N. Zilberman, "Planter: Rapid prototyping of in-network machine learning inference," *ACM SIGCOMM Computer Communication Review*, vol. 54, no. 1, pp. 2–21, 2024.

[6] S. Mittal, H. V, P. Heetkumar, and P. Tammana, "iGuard: Efficient isolation forest design for malicious traffic detection in programmable switches," in *Proceedings of the 20th International Conference on emerging Networking EXperiments and Technologies*, 2024, pp. 55–64.

[7] H. Yan, Q. Li, G. Xie, G. Tyson, and Y. Jiang, "Linc: Enabling low-resource in-network classification and incremental model update," in *2024 IEEE 32nd International Conference on Network Protocols (ICNP)*. IEEE, 2024, pp. 1–12.

[8] G. Siracusano, S. Galea, D. Sanvito, M. Malekzadeh, G. Antichi, P. Costa, H. Haddadi, and R. Bifulco, "Re-architecting traffic analysis with neural network interface cards," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 513–533.

[9] M. Zhang, L. Cui, X. Zhang, F. P. Tso, Z. Zhen, Y. Deng, and Z. Li, "Quark: Implementing convolutional neural networks entirely on programmable data plane," in *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 2025, pp. 1–10.

[10] S. Schwarzmann, T. E. Civelek, A. Iera, D. Corujo, G. T. Karetsos, R. Guerzoni, O. Abboud, A. M. Valenzuela, R. Trivisonno, M. G. Spina *et al.*, "Native support of ai applications in 6g mobile networks via an intelligent user plane," in *2024 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 2024, pp. 1–6.

[11] J. Yan, H. Xu, Z. Liu, Q. Li, K. Xu, M. Xu, and J. Wu, "Brain-on-Switch: Towards advanced intelligent network data plane via NN-Driven traffic analysis at Line-Speed," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024, pp. 419–440.

[12] K. Zhang, N. Samaan, and A. Karmouch, "A machine learning-based toolbox for p4 programmable data-planes," *IEEE Transactions on Network and Service Management*, vol. 21, no. 4, pp. 4450–4465, 2024.

[13] D. Wen, Z. Liu, T. Yang, T. Li, T. Li, C. Li, J. Li, and Z. Sun, "Inference-to-complete: A high-performance and programmable data-plane co-processor for neural-network-driven traffic analysis," *arXiv preprint arXiv:2411.00408*, 2024.

[14] C. Zheng, X. Hong, D. Ding, S. Vargaftik, Y. Ben-Itzhak, and N. Zilberman, "In-network machine learning using programmable network devices: A survey," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 2, pp. 1171–1200, 2023.

[15] Z. Wen and G. Yan, "HiP4-UPF: Towards High-Performance comprehensive 5g user plane function on p4 programmable switches," in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, 2024, pp. 303–320.

[16] F. Petersen, H. Kuehne, C. Borgelt, J. Welzel, and S. Ermon, "Convolutional differentiable logic gate networks," *Advances in Neural Information Processing Systems*, vol. 37, pp. 121 185–121 203, 2024.

[17] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen, "Deep differentiable logic gate networks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 2006–2018, 2022.

[18] A. Agrawal and C. Kim, "Intel tofino2–a 12.9 tbps p4-programmable ethernet switch," in *2020 IEEE Hot Chips 32 Symposium (HCS)*. IEEE Computer Society, 2020, pp. 1–32.

[19] N. McKeown, "Pisa: Protocol independent switch architecture," in *P4 Workshop*, vol. 516, 2015.

[20] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese *et al.*, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[21] Barefoot Networks, "Tofino native architecture (tna) overview," https://raw.githubusercontent.com/barefootnetworks/Open-Tofino/master/PUBLIC_Tofino-Native-Arch.pdf, Barefoot Networks, Tech. Rep., 2021, accessed: 2025-08-13.

[22] Z. Zhao, Z. Li, Z. Song, F. Zhang, and B. Chen, "Rids: Towards advanced ids via rnn model and programmable switches co-designed approaches," in *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 2024, pp. 591–600.

[23] A. Bas, "Leveraging stratum and tofino fastrefresh for software upgrades," Barefoot Networks, Tech. Rep., 2018, accessed: Aug. 13, 2025. [Online]. Available: https://opennetworking.org/wp-content/uploads/2018/12/Tofino_Fast_Refresh.pdf

[24] K. Wiles, "Pktgen-dpdk: Dpdk-based packet generator," https://pktgen-dpdk.readthedocs.io/, 2025, accessed: 2025-08-14.

[25] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *2015 military communications and information systems conference (MilCIS)*. IEEE, 2015, pp. 1–6.

[26] T. Benson, A. Akella, and D. A. Maltz, "Network traffic characteristics of data centers in the wild," in *Proceedings of the 10th ACM SIGCOMM conference on Internet measurement*, 2010, pp. 267–280.